

A Comparative Evaluation of Sentiment Analysis Models

This project explores the development and evaluation of a cloud-based sentiment analysis microservice designed to classify short texts as positive, negative, neutral, or mixed. The project was motivated by the need for a lightweight yet functional tool capable of offering sentiment insights in real time. The service was implemented using Amazon Web Services (AWS), specifically AWS Lambda for serverless computers and API Gateway for managing HTTP requests. AWS Comprehend served as the underlying NLP engine. The model's performance was compared against Grammarly, ChatGPT (GPT-4), and a custom machine learning model trained using multinomial logistic regression.

While grammar feedback was initially part of the project's vision, time and resource limitations made it unfeasible within the scope of this course. However, it remains a key future enhancement. Long-term goals also include integrating behavioral psychology-based feedback, allowing the tool to adapt its responses to users' writing styles or psychological profiles.

DATA OVERVIEW

To train the custom ML model, we compiled a unified dataset named `sentiment_data.csv`. This dataset was created by merging three distinct sources:

1. **Sentiment140**: Containing tweets labeled as positive, negative, or neutral, commonly used for sentiment classification tasks.

2. Sentiment Analysis by mdismielhossenabir: Provided additional labeled examples in the same categories.
3. Synthetic Data: Generated to represent the "mixed" sentiment class, which was not available in the above datasets. These samples were either written by the project author or derived from previously annotated examples.

The merged dataset presented challenges related to class imbalance. Initially, the machine learning model was biased toward negative sentiment predictions. To mitigate this, a combination of class reweighting and sampling strategies were implemented, including oversampling of minority classes and downsampling of overrepresented classes. These adjustments helped balance the dataset and enhance the model's ability to generalize.

The combined dataset was stored as `sentiment_data.csv`, and is included in the project files. Additionally, an API was deployed to host the AWS-based microservice:

- Endpoint: <https://muoen4uszc.execute-api.us-west-2.amazonaws.com/sentiment-prod>
- Method: POST
- Payload Format: JSON

Example request body:

```
{
```

```
"text": "Your text here."  
}
```

This setup allows for easy testing and integration into other applications by returning sentiment results in a structured JSON format.

ANALYTICS PERFORMED

To assess the performance of our tool, we tested four models on two writing samples: an original mythological passage and an excerpt from *Percy Jackson & the Olympians*. The following platforms were used:

1. ChatGPT (GPT-4)
2. Grammarly
3. AWS Comprehend (via our API)
4. Custom Multinomial Logistic Regression Model

The machine learning model was trained using the combined dataset. Preprocessing included cleaning, tokenization, normalization, and one-hot encoding. We used a typical 70-15-15 train-validation-test split. To combat class imbalance, we experimented with resampling techniques and assigned balanced class weights during model training.

Each sentence from the test sets was evaluated by all four models. The results were compiled into tables showing sentiment predictions across platforms. This structure enabled

direct comparison between models, highlighting discrepancies and patterns in sentiment interpretation.

RESULTS

The comparative evaluation demonstrated that ChatGPT provided the most contextually nuanced predictions, particularly excelling at detecting mixed sentiments. Grammarly also handled sentiment effectively but leaned more toward grammatical rather than emotional accuracy. AWS Comprehend was fast and reliable with clear, literal language but showed limitations when faced with figurative or poetic passages.

Our ML model initially skewed heavily toward negative classifications due to the original class imbalance. After applying class weighting and synthetic oversampling techniques, its performance improved substantially. It was able to accurately classify positive, neutral, and some mixed sentiments, though it still lagged behind ChatGPT in interpretive nuance.

Overall, the microservice showed practical strength for basic classification tasks and demonstrated promise for more advanced usage with further tuning.

CONCLUSION AND FUTURE DIRECTIONS

In conclusion, this project successfully implemented a functional, cloud-based sentiment analysis microservice and evaluated it against leading NLP tools. While grammar feedback could not be included in this iteration, it remains a priority for future development. The API

infrastructure proved reliable and scalable, and the machine learning model, once balanced, provided reasonable performance.

Future improvements will include (1) Grammar and tone analysis, potentially using spaCy or additional AWS features; (2) Enhanced modeling for abstract and creative writing styles; (3) Integration of psychological profiling to personalize writing feedback.

Ultimately, this project offers a solid, working prototype that connects cloud architecture, machine learning, and NLP for practical and scalable sentiment analysis. Its future development will aim to refine both its analytical accuracy and the depth of feedback provided to end users.

Resources

Sentiment140 Dataset

Go, Alec, Richa Bhayani, and Lei Huang. *Twitter Sentiment Classification using Distant Supervision*. Stanford University, 2009,
<https://www.kaggle.com/datasets/kazanova/sentiment140>.

Sentiment Analysis Dataset

Hossen Abir, Md. Ismiel. *Sentiment Analysis*. Kaggle,
<https://www.kaggle.com/datasets/mdismielhossenabir/sentiment-analysis>.

Synthetic Data

User-generated synthetic data created for class balancing and “mixed” sentiment augmentation. Not published externally.

ChatGPT (GPT-4)

OpenAI. *ChatGPT*. 2024 version, <https://chat.openai.com/>.

Grammarly

Grammarly Inc. *Grammarly AI Writing Assistant*, <https://www.grammarly.com/>.

AWS Comprehend

Amazon Web Services, Inc. *Amazon Comprehend*. <https://aws.amazon.com/comprehend/>.

Custom ML Model (Multinomial Logistic Regression)

Trained by the authors using Python's scikit-learn library on a combined dataset.

AWS Deployment Tools

AWS Lambda. Amazon Web Services, <https://aws.amazon.com/lambda/>.

Amazon API Gateway. Amazon Web Services, <https://aws.amazon.com/api-gateway/>.